

Síntesis · Día 3 — Generar, entregar, acumular

El día cambia de naturaleza dos veces: la primera mitad trabaja un tipo de herramienta nuevo (generadores de ejercicios, ya no visualizaciones), y el cierre es institucional (dónde va a vivir todo lo que se produce en el departamento).

De visualización a generación

Hasta el Día 2 se construyeron herramientas que **acompañan** una explicación: el alumno observa o manipula una representación. El Día 3 introduce los **generadores**: programas que producen ejercicios nuevos (con solución) cada vez que se ejecutan — el tipo de trabajo que normalmente se escribe a mano, cada semestre, otra vez.

Esto se hace en **Google Colab con Python**, porque la librería **sympy** calcula simbólicamente (deriva, integra y resuelve de forma exacta, no aproximada), y eso es exactamente lo que se necesita para que la solución de cada ejercicio generado sea confiable.

Los tres componentes de un generador (los mismos cinco elementos del Día 1, adaptados)

1. **Espacio de parámetros** — qué varía entre una versión y otra, y en qué rango. Debe ser matemáticamente intencional, no arbitrario:
 - *Mal*: "con números aleatorios" (no dice rango, ni si son enteros, ni restricciones).
 - *Bien*: "coeficientes enteros entre -10 y 10, excluyendo el cero en el coeficiente principal, de forma que el polinomio siempre sea efectivamente de grado 3."
 - Un rango demasiado amplio produce ejercicios sin solución limpia; uno demasiado angosto genera ejercicios que se repiten y el alumno reconoce el patrón sin practicar el método.
2. **Formato del enunciado** — cómo se presenta al alumno. En Colab, se pide notación **LaTeX** ($\dots$) para que se vea como una fórmula real, no como texto plano con asteriscos.
3. **Solución verificable** — el equivalente al Elemento 5. **Regla de oro del día**: si el prompt no pide explícitamente "*solución paso a paso*" o "*procedimiento completo*", la IA entrega solo la respuesta final. Eso sirve para calificar, pero no sirve para que el alumno aprenda del proceso ni para validar (si solo hay un número, no hay nada que verificar término por término).

Cómo se valida un generador

Igual que el Día 2, pero aplicado a un ejercicio generado: se elige uno de los ejercicios producidos, se toma un término del resultado, se verifica a mano que la operación sea correcta. **El hecho de usar sympy no exime de validar** — sympy deriva bien, pero todo lo demás (que el enunciado impreso coincida con lo derivado, que el formato no pierda un término) puede fallar y hay que revisarlo.

Cómo entregarlo al alumno — tres vías

Vía	Cuándo conviene	Detalle
Archivo descargable (PDF con varias corridas compiladas, título y fecha)	Tarea o examen	Se ejecuta el generador varias veces, se copia la salida, se da formato y se exporta. Es la vía más simple.
Enlace al notebook (permiso de solo lectura)	Práctica libre	El alumno hace su propia copia y ejecuta cuantas veces quiera. Cada alumno necesita cuenta Google.
Incrustado en el LMS	Integración al curso	Añade un paso de formato cada vez que se actualiza.

La pregunta que suele decidir la vía sola: **¿es para tarea, práctica libre o examen?** Un examen no se entrega como generador vivo (se necesitan versiones fijas y distintas por alumno o fila).

Los cinco criterios de "terminado", aplicados a un generador

Los mismos del Día 2: corre sin errores (ejecutar la celda tres veces), validado a mano (un ejercicio verificado término por término), interactividad (cada ejecución produce un resultado distinto), comentado con título comprensible, y se sabe cómo compartirlo (el documento existe y abre en otro dispositivo).

El repositorio del departamento — qué es, y qué no hace falta todavía

Todo lo que se construye durante la semana va a vivir en un espacio **que no es una carpeta compartida más**: guarda todas las versiones anteriores con fecha y autor, así que nunca se pierde ni se sobrescribe accidentalmente una versión previa.

Hay **tres espacios**, cada uno para una audiencia distinta:

- **GitHub** — para el profesor que contribuye o edita una herramienta: necesita historial de cambios.
- **Drive institucional** — para los materiales del taller mismo (blueprints, actas), que no son código.
- **Canal del alumno** — el alumno solo necesita un enlace; no interactúa con GitHub ni con Drive.

Importante: el Día 3 solo presenta la idea. No hace falta crear ninguna cuenta, no hace falta subir nada, y **no se instala nada, nunca** — se usa desde el navegador. El repositorio se crea formalmente el Día 5, y para entonces cada participante ya tiene una contribución ahí (por los enlaces de herramientas que se van registrando desde el Día 1).

Esto también prepara la **decisión colegiada del Día 5**: parte de esa decisión es qué queda en el repositorio y quién lo mantiene. No se decide el Día 3; se decide el viernes con la experiencia acumulada de toda la semana.

Producto del día

Un generador de ejercicios de la materia propia, validado a mano, con su rango ajustado (casi ningún generador sale bien calibrado al primer intento — hay que ejecutarlo varias veces y ajustar los límites), y un documento con varias corridas compiladas, con título y fecha, listo para repartir.

Tarea entre Día 3 y Día 4

Ninguna tarea formal. Solo se conservan los enlaces de las herramientas y el documento de ejercicios del día — se necesitan más adelante.

Términos clave del día

- **Generador** (de ejercicios) — programa que produce versiones nuevas de un mismo tipo de ejercicio, con solución.
- **Espacio de parámetros** — el componente 1 de un generador: qué varía y en qué rango.
- **Repositorio de materiales del departamento** — el espacio con historial de versiones donde vive todo lo producido; no es una carpeta de Drive.
- **README** — el documento que describe qué hace una herramienta y su caso de validación, para que otro profesor la entienda sin preguntar directamente.